GPU并行计算课程实验报告（五次小实验汇总）

学生姓名： 裴宇皓
学号： SA25006135

实验一：OpenMP 及 CUDA 实验环境搭建

1.1 实验目的

完成并行计算开发环境的配置，确认编译器与硬件设备正常运行。

1.2 软硬件环境配置

- **CPU**: Intel Core i7-14650HK
- **GPU**: NVIDIA GeForce RTX 5060 (8GB, 80W)
- **OS**: Windows 11
- **Compiler**: NVCC (CUDA 12.8), MSVC (Visual Studio)
- **IDE**：Visual Studio 2022
- **编译器/SDK**：
  - **CUDA**：Toolkit 12.8 (Driver 573.24)
  - **OpenMP**：MSVC 自带支持 (OpenMP 2.0 Standard)，同时利用 -openmp:llvm 启用 LLVM 的 OpenMP 运行

实验二：排序算法的并行及优化 (OpenMP)

2.1 实验内容与算法选择

- **算法选择**：选择奇偶排序，适合并行化。
- **数据规模**：设定固定随机种子生成的10万个元素。

2.2 串行实现代码

奇偶排序是冒泡排序的一种变体，主要特点是将整个排序过程分解为两个交替进行的阶段，分别比较并交换所有偶数索引和奇数索引及其相邻元素的对。

代码实现逻辑： 串行程序使用一个 `while` 循环控制整体迭代，内部通过标志位 `sorted` 来检测数组是否有序。每一轮迭代中，程序依次执行一次偶数下标遍历和一次奇数下标遍历。如果在某一轮中，两个阶段均未发生任何数据交换（即 `sorted` 保持为 1），则说明数组已完全有序，算法终止。

```
// 串行版本的奇偶排序
void odd_even_sort_serial(int arr[], int n) {
    int sorted = 0;
    int temp;
    while (!sorted) {
        sorted = 1;  // 先假设数组已经排好了
        // 偶阶段：处理下标 0, 2, 4...
```

```
        for (int i = 0; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                sorted = 0;  // 发生了交换，说明还没完全有序
            }
        }
        // 奇阶段：处理下标 1, 3, 5...
        for (int i = 1; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                sorted = 0;
            }
        }
    }
}
```

2.3 并行实现代码

针对奇阶段和偶阶段，并行代码中使用 `#pragma omp parallel for` 指令将原本串行的 `for` 循环自动拆解，分配给 N 个线程同时执行。代码中使用了原子操作 `#pragma omp atomic write` 用于记录全局是否存在位置交换，即排序是否完成。在并行区域内，交换元素所需的临时变量 `temp` 被显式声明为 `private`，防止出现多线程的并发冲突。`volatile` 关键字定义 `global_sorted` 强制编译器对该变量的每一次读取都直接从主内存进行，防止出现变量的值没能被正确读取。

```
// 并行版本的奇偶排序（OpenMP实现）
void odd_even_sort_parallel(int arr[], int n) {
    int sorted = 0;
    int temp;
    // 使用 volatile 确保多线程读取时是最新的值
    volatile int global_sorted = 1;
    while (!sorted) {
        // 每一轮开始前重置标志
        global_sorted = 1;
        // 并行处理偶阶段
        // private(temp) 确保每个线程有自己的临时变量，防止冲突
#pragma omp parallel for private(temp)
        for (int i = 0; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
                // 使用原子操作修改全局标志，只要有交换发生就置为0
#pragma omp atomic write
                global_sorted = 0;
            }
        }
        // 并行处理奇阶段
#pragma omp parallel for private(temp)
        for (int i = 1; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                temp = arr[i];
                arr[i] = arr[i + 1];
```

```
                arr[i + 1] = temp;
#pragma omp atomic write
                global_sorted = 0;
            }
        }
        // 读取本轮排序后的状态
#pragma omp atomic read
        sorted = global_sorted;
    }
}
```

2.4 实验结果

设置不同的线程进行测试，实验电脑支持的最大线程数设置为 24。

```
正在运行串行版本 (数据量 100000)...
串行版本完成，耗时: 9.4552 秒

=== 并行加速比测试 ===
线程数          耗时(秒)        加速比          结果状态
--------------------------------------------------------
2               9.0887          1.04 x          通过
4               7.8805          1.20 x          通过
8               8.5945          1.10 x          通过
12              10.3643         0.91 x          通过
16              10.8065         0.87 x          通过
24              11.8030         0.80 x          通过
```

结果如上，可以看到并不是线程数越多越好，因为启用更多的线程也带来更多的时间开销。由于`#pragma omp parallel for`在循环结束时包含隐式同步点，以及原子操作竞争等原因，加速比的提升并不高也是可以理解的。

实验三：矩阵乘法的并行及优化(OpenMP)

3.1 实验内容与算法选择

- **算法选择**：完成矩阵乘法运算 $C = A \times B$ 的 OpenMP并行。
- **数据规模**：设定固定随机种子生成两个 1024 维度的方阵进行相乘计算，涉及约 20 亿次浮点运算，串行执行耗时较长。

3.2 串行实现代码

串行版本严格遵循矩阵乘法的数学定义，采用三重循环结构：

1. 外两层循环遍历结果矩阵的每一个坐标位置。
2. 最内层循环计算 A 的第 i 行与 B 的第 j 列的点积。

```
// 串行矩阵乘法
void matrix_multiply_serial(const double* A, const double* B, double* C, int N)
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double sum = 0.0;
            // 标准的 i-j-k 三重循环
            for (int k = 0; k < N; k++) {
                sum += A[i * N + k] * B[k * N + j];
            }
            C[i * N + j] = sum;
        }
    }
}
```

3.3 并行实现代码

使用OpenMP进行并行化，使用 `#pragma omp parallel for collapse(2)` 将原本串行代码的外两层循环合并。由于`sum` 在循环内定义，因此利用了每个线程的寄存器，可以安全累加。

```
// 并行矩阵乘法
void matrix_multiply_parallel(const double* A, const double* B, double* C, int
N) {
    // 采用 collapse(2) 将两层循环合并，增加并行任务粒度
#pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double sum = 0.0;
            // 内层 k 循环保持串行，sum 为局部变量防止竞争
            for (int k = 0; k < N; k++) {
                sum += A[i * N + k] * B[k * N + j];
            }
            C[i * N + j] = sum;
        }
    }
}
```

3.4 实验结果

设置不同的线程进行测试，因为该实验中并行版本的运行时间比较短，因此采用了多次测量取平均值的做法。实验结果如下

```
=== 矩阵乘法实验 (OpenMP) ===
矩阵规模: 1024 x 1024
测量模式: 每轮运行 5 次取平均值
------------------------------------------------
正在运行串行基准测试...

=== 实验结果统计 ===
| Threads    | Avg Time(s)  | Speedup    |
|------------|--------------|------------|
| Serial     | 4.8981       | -          |
| 4          | 1.6980       | 2.88       x |
| 8          | 0.9776       | 5.01       x |
| 12         | 0.7022       | 6.98       x |
| 16         | 0.4304       | 11.38      x |
| 24         | 0.3464       | 14.14      x |
```

可以看到，并行有效提升了运算速度，并且当线程数为24时，加速比最大，这说明此时任务还没有因为内存带宽的原因速度受限，提升线程数一直保持有效加速。

实验四：快速傅里叶变换的并行实现(CUDA)

4.1 实验内容与算法介绍

- **算法选择**：本实验采用经典的 **Cooley-Tukey 迭代算法**，其流程可以概括为：位反序置换、分治迭代以及蝶形运算等步骤。
- **数据规模**：选择固定的随机种子，生成信号长度$N = 2^{20}$ （即 1,048,576 点），数据类型为 Double，此时该任务为一个计算密集型任务，CUDA并行加速可以显著提高运行速度。

4.2 串行实现代码

数据重排（二进制倒序）

在开始计算前，需要调整输入数组的数据位置。

- 方法：把数组下标转成二进制，然后首尾颠倒。例如对于长度为 8 的数组，下标 1 的二进制是 `001`，倒过来是 `100`（即下标 4），于是要把数组第 1 个数和第 4 个数交换位置。
- 目的：完成重排后，后续的计算可以在原数组上直接进行，不需要申请额外的内存空间。

蝶形运算

重排完成后，进行多轮迭代蝶形运算。

- **分级计算**：如果数组长度是 N，一共需要进行 $\log_2 N$ 级运算。
- **分组处理**：每一级运算中，数据被分成若干组。每一级的分组大小都翻倍（2 -> 4 -> 8 -> ... -> N），直到最后一级将所有数据看作一个大组进行运算。分组示例如下：
  第一级运算：将相邻的 **2** 个数据看作一组（如第 0、1 个，第 2、3 个...），组内进行蝶形运算。此时两个参与运算的数据间隔（步长）为 1。
  第二级运算：将计算好的数据两两合并，每 **4** 个数据看作一组（如第 0~3 个，第 4~7 个...），此时前半部分（第 0、1 个）分别与后半部分（第 2、3 个）进行运算，数据间隔为 2。
  以此类推
- **核心公式**：在每一组内部，通过旋转因子 W （$W = e^{-i\frac{2\pi}{L}k}$）将两个数据进行混合。对于两个输入点 A 和 B，计算出两个新值：
  - 新值 1 = $A + W \times B$
  - 新值 2 = $A - W \times B$

通过迭代，不断的把长序列分解成短序列处理，最终得到了频域的结果。

```
// ==========================================
// 串行版本 (CPU)
// ==========================================

// 辅助函数：计算二进制反转
int reverse_bits(int x, int logN) {
    int res = 0;
    for (int i = 0; i < logN; i++) {
        if ((x >> i) & 1) {
            res |= (1 << (logN - 1 - i));
        }
    }
    return res;
}

// 串行 FFT 算法
void fft_serial(Complex_d* data, int N) {
    if (N == 0) return;
    int logN = 0;
    while ((1 << logN) < N) logN++;
    // 位反序重排
    for (int i = 0; i < N; i++) {
        int rev_i = reverse_bits(i, logN);
        if (i < rev_i) {
            std::swap(data[i], data[rev_i]);
        }
    }
    // 迭代进行蝶形运算
    for (int len = 2; len <= N; len <<= 1) {
        double angle = -2.0 * M_PI / len;
        Complex_d w(cos(angle), sin(angle));
        for (int i = 0; i < N; i += len) {
            Complex_d w_k(1.0, 0.0);
            for (int k = 0; k < len / 2; k++) {
                Complex_d u = data[i + k];
                Complex_d v = data[i + k + len / 2] * w_k;
                data[i + k] = u + v;
                data[i + k + len / 2] = u - v;
                w_k *= w;
            }
        }
    }
}
```

4.3 并行实现代码

本实验通过两个 Kernel 函数将串行 FFT 算法映射到 GPU 的线程网格中。

位反序重排 Kernel：

将 N 个数据点直接映射给 N 个 GPU 线程。每个线程通过自己的全局索引 `idx`，独立计算出对应的二进制反序索引。根据计算出的反序索引，进行数据交换，代码中的判断条件 `if (idx < rev_idx)` 会保值不重复交换。

```
// 核函数：位反序重排
```

```
__global__ void bit_reverse_kernel(Complex_d_cuda* d_data, int N, int logN) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        int rev_idx = 0;
        int temp = idx;
        for (int i = 0; i < logN; i++) {
            rev_idx = (rev_idx << 1) | (temp & 1);
            temp >>= 1;
        }
        if (idx < rev_idx) {
            Complex_d_cuda temp_val = d_data[idx];
            d_data[idx] = d_data[rev_idx];
            d_data[rev_idx] = temp_val;
        }
    }
}
```

蝶形运算 Kernel

在 FFT 的每一级迭代中，共有 $N/2$ 次蝶形运算。我们将这 $N/2$ 个运算任务分配给 GPU 线程，每个线程只负责计算一对元素的更新，即每个线程负责一对蝶形更新。线程`idx`和线程负责数据之间的映射关系如下：
`group`：计算当前线程负责第几组数据块；`pos_in_group`：计算当前线程在组内的相对位置。利用这两个参数算出实际的数据索引`i`和`k`，从而准确定位到参与运算的两个元素 $X[i+k]$ 和 $X[i+k+len/2]$

```
// 核函数：蝶形运算
__global__ void butterfly_kernel(Complex_d_cuda* d_data, int N, int len) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int group = idx / (len / 2);      // 第几组
    int pos_in_group = idx % (len / 2); // 组内第几个
    // 越界检查
    if (group * len + pos_in_group < N) {
        int i = group * len;
        int k = pos_in_group;
        // 计算旋转因子
        double angle = -2.0 * M_PI * k / len;
        Complex_d_cuda w = make_double2(cos(angle), sin(angle));
        // 读取数据
        Complex_d_cuda u = d_data[i + k];
        Complex_d_cuda v = d_data[i + k + len / 2];
        // 复数乘法: v * w
        double real_part = v.x * w.x - v.y * w.y;
        double imag_part = v.x * w.y + v.y * w.x;
        Complex_d_cuda vw = make_double2(real_part, imag_part);
        // 蝶形更新
        d_data[i + k] = make_double2(u.x + vw.x, u.y + vw.y);
        d_data[i + k + len / 2] = make_double2(u.x - vw.x, u.y - vw.y);
    }
}
```

并行FFT启动函数

```
// CUDA FFT 启动函数
void fft_cuda(Complex_d_cuda* d_data, int N) {
    int logN = log2_int(N);
```

```
    // 位反序
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    bit_reverse_kernel << <blocksPerGrid, threadsPerBlock >> > (d_data, N,
logN);
    cudaDeviceSynchronize();
    // 多级蝶形运算
    for (int len = 2; len <= N; len <<= 1) {
        int threadsPerLayer = N / 2; // 总共有 N/2 个蝶形运算
        blocksPerGrid = (threadsPerLayer + threadsPerBlock - 1) /
threadsPerBlock;
        butterfly_kernel << <blocksPerGrid, threadsPerBlock >> > (d_data, N,
len);
        cudaDeviceSynchronize();
    }
}
```

可以看到针对在迭代计算蝶形运算的时候，随着循环的进行，`butterfly_kernel` 被多次启动。

4.4 实验结果

采用了多次测量取平均值的做法。实验结果如下

```
=== FFT 并行加速实验 ===
信号长度: 1048576
测量模式: 运行 5 次取平均
----------------------------------------
正在运行并行版本 (Warm-up included)...
并行计算完成。
正在运行串行版本...
串行计算完成。

=== 实验结果统计 ===
| Version      | Avg Time(s)  | Speedup     |
|--------------|--------------|-------------|
| Serial       | 0.1132       | -           |
| Parallel     | 0.0051       | 22.12      x |

(结果验证: 通过)
```

可以看到串行版本在CPU上也运算得非常快，这是因为FFT算法本身非常巧妙，成功将原本普通DFT的复杂度 $O(N^2)$降低到了$O(N\log N)$，并且该算法的缓存命中率极高，这就使得的本身FFT的串行版本在处理如此大规模的数据时也效率极高。在此基础上使用 CUDA 并行化的FFT依然可以取得22倍的加速比。

调整核函数启动时设定的 block 中的 thread 数，实验结果发现加速比变化不大，最终选择每个 block 中设置256个 thread 。

4.5 计时补充说明

以上并行代码计时时没有纳入数据传输的过程，仅展示了纯粹的计算时间。针对 CUDA并行 采用了专门的GPU事件计时。

```
cudaEventRecord(start_event, 0);
fft_cuda(d_data, N_TEST);
cudaEventRecord(end_event, 0);
cudaEventSynchronize(end_event);
```

实验五：图像卷积算法的并行及优化(CUDA)

5.1 实验内容与算法介绍

- **算法选择**：本实验采用固定的图像卷积核，在图像上进行卷积操作。卷积操作相当于是对于图像中的每个像素，计算其以及周围邻域像素与卷积核权重的加权和。
- **数据规模**：输入为 4096 x 4096 的 RGB 图像，使用 3x3 卷积核进行处理。
- **数据准备**：生成一张“棋盘图”，使用高斯模糊核。

```
// 初始化图像
printf("正在初始化图像 (%d x %d)...\n", WIDTH, HEIGHT);
for (int i = 0; i < WIDTH * HEIGHT; i++) {
    int pattern = ((i % WIDTH / 64) + (i / WIDTH / 64)) % 2;
    h_input[i * 3 + 0] = pattern ? 255 : 0;
    h_input[i * 3 + 1] = pattern ? 255 : 0;
    h_input[i * 3 + 2] = pattern ? 255 : 0;
}
//卷积核
float h_kernel[] = {
    0.0625f, 0.125f, 0.0625f,
    0.125f,  0.25f,  0.125f,
    0.0625f, 0.125f, 0.0625f
};
```

5.2 串行实现代码

为了计算每一个像素的新值，串行算法采用了四层嵌套循环，外层两重循环遍历了图像的每一个像素坐标，内层两层循环针对每一个卷积核滑动到的位置，计算原图像与卷积核的加权和。代码中通过设定循环范围，`pad` 到 `width - pad`，忽略了图像最边缘无法被卷积核完全覆盖的像素，避免越界风险。对卷积后的累加结果做了数值阶段，保证数值在[0, 255]之间，使得可以用`unsigned char`类型存储。

```
void convolution_cpu(unsigned char* input, unsigned char* output, float* kernel,
int width, int height, int k_size) {
    int pad = k_size / 2;
    // 遍历每一个像素
    for (int y = pad; y < height - pad; y++) {
        for (int x = pad; x < width - pad; x++) {
            float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;
            // 遍历卷积核
            for (int ky = -pad; ky <= pad; ky++) {
```

```
                for (int kx = -pad; kx <= pad; kx++) {
                    int img_x = x + kx;
                    int img_y = y + ky;

                    int pixel_idx = (img_y * width + img_x) * CHANNELS;
                    int kernel_idx = (ky + pad) * k_size + (kx + pad);

                    float weight = kernel[kernel_idx];

                    sum_r += input[pixel_idx] * weight;
                    sum_g += input[pixel_idx + 1] * weight;
                    sum_b += input[pixel_idx + 2] * weight;
                }
            }
            // 写入结果
            int out_idx = (y * width + x) * CHANNELS;
            output[out_idx] = (unsigned char)fminf(fmaxf(sum_r, 0.0f), 255.0f);
            output[out_idx + 1] = (unsigned char)fminf(fmaxf(sum_g, 0.0f),
255.0f);
            output[out_idx + 2] = (unsigned char)fminf(fmaxf(sum_b, 0.0f),
255.0f);
        }
    }
}
```

5.3 并行实现代码

将卷积计算并行化，利用 CUDA 的二维线程网格（Grid）和线程块（Block）结构，将 GPU 中的每一个线程唯一映射到输出图像的一个像素坐标。代码通过 `x = blockIdx.x * blockDim.x + threadIdx.x` 和 `y =blockIdx.y * blockDim.y + threadIdx.y` 计算出当前线程在全局图像中的绝对坐标，之后完成该位置的加权计算。

```
//cuda并行卷积计算
__global__ void convolution_global_kernel(unsigned char* input, unsigned char*
output, float* kernel, int width, int height, int k_size) {
    int pad = k_size / 2;
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= pad && x < width - pad && y >= pad && y < height - pad) {
        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;
        for (int ky = -pad; ky <= pad; ky++) {
            for (int kx = -pad; kx <= pad; kx++) {
                int img_x = x + kx;
                int img_y = y + ky;
                // 普通读取：每次都去全局显存拿数据，慢
                float weight = kernel[(ky + pad) * k_size + (kx + pad)];
                int idx = (img_y * width + img_x) * CHANNELS;
                sum_r += input[idx] * weight;
                sum_g += input[idx + 1] * weight;
                sum_b += input[idx + 2] * weight;
            }
        }
        int out_idx = (y * width + x) * CHANNELS;
        output[out_idx] = (unsigned char)fminf(fmaxf(sum_r, 0.0f), 255.0f);
        output[out_idx + 1] = (unsigned char)fminf(fmaxf(sum_g, 0.0f), 255.0f);
```

```
        output[out_idx + 2] = (unsigned char)fminf(fmaxf(sum_b, 0.0f), 255.0f);
    }
}
```

在以上版本中，卷积核权重存放在 GPU 的全局内存中。由于卷积核是固定的，并且每个线程都需要访问，因此为了减小带宽压力，尝试使用常量内存存储卷积核。这时当线程访问常量内存中的卷积核时，可以充分利用线程束（warp）的广播机制，极大地缓解内存瓶颈。

```
//常量内存优化并行卷积计算
__global__ void convolution_constant_kernel(unsigned char* input, unsigned char*
output, int width, int height, int k_size) {
    int pad = k_size / 2;
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= pad && x < width - pad && y >= pad && y < height - pad) {
        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

        for (int ky = -pad; ky <= pad; ky++) {
            for (int kx = -pad; kx <= pad; kx++) {
                int img_x = x + kx;
                int img_y = y + ky;
                // 优化读取：直接从常量缓存拿数据，快
                float weight = c_kernel_storage[(ky + pad) * k_size + (kx +
pad)];

                int idx = (img_y * width + img_x) * CHANNELS;
                sum_r += input[idx] * weight;
                sum_g += input[idx + 1] * weight;
                sum_b += input[idx + 2] * weight;
            }
        }

        int out_idx = (y * width + x) * CHANNELS;
        output[out_idx] = (unsigned char)fminf(fmaxf(sum_r, 0.0f), 255.0f);
        output[out_idx + 1] = (unsigned char)fminf(fmaxf(sum_g, 0.0f), 255.0f);
        output[out_idx + 2] = (unsigned char)fminf(fmaxf(sum_b, 0.0f), 255.0f);
    }
}
```

```
//运行以下代码将卷积核数据存储到常量内存中
cudaMemcpyToSymbol(c_kernel_storage, h_kernel, k_bytes);
```

5.4 实验结果

采用了以下函数测试两个版本 CUDA kernel的运行时间，并和串行版本比较计算加速比。

```
// GPU 测试辅助函数
double run_gpu_test(int mode, unsigned char* h_input, unsigned char* h_output,
float* h_kernel, int k_size) {
    unsigned char* d_input, * d_output;
    float* d_kernel = NULL;
    size_t img_bytes = WIDTH * HEIGHT * CHANNELS * sizeof(unsigned char);
    size_t k_bytes = k_size * k_size * sizeof(float);
```

```
    cudaMalloc(&d_input, img_bytes);
    cudaMalloc(&d_output, img_bytes);
    cudaMemcpy(d_input, h_input, img_bytes, cudaMemcpyHostToDevice);

    if (mode == 0) { // 全局内存模式
        cudaMalloc(&d_kernel, k_bytes);
        cudaMemcpy(d_kernel, h_kernel, k_bytes, cudaMemcpyHostToDevice);
    }
    else { // 常量内存模式
        cudaMemcpyToSymbol(c_kernel_storage, h_kernel, k_bytes);
    }

    dim3 block(16, 16);
    dim3 grid((WIDTH + block.x - 1) / block.x, (HEIGHT + block.y - 1) /
block.y);

    // 预热
    if (mode == 0) convolution_global_kernel << <grid, block >> > (d_input,
d_output, d_kernel, WIDTH, HEIGHT, k_size);
    else convolution_constant_kernel << <grid, block >> > (d_input, d_output,
WIDTH, HEIGHT, k_size);
    cudaDeviceSynchronize();

    // 计时
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    float total_time = 0.0f;
    for (int i = 0; i < MEASURE_RUNS; i++) {
        cudaEventRecord(start);
        if (mode == 0) convolution_global_kernel << <grid, block >> > (d_input,
d_output, d_kernel, WIDTH, HEIGHT, k_size);
        else convolution_constant_kernel << <grid, block >> > (d_input,
d_output, WIDTH, HEIGHT, k_size);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float ms = 0;
        cudaEventElapsedTime(&ms, start, stop);
        total_time += ms;
    }

    cudaMemcpy(h_output, d_output, img_bytes, cudaMemcpyDeviceToHost);

    cudaFree(d_input); cudaFree(d_output);
    if (d_kernel) cudaFree(d_kernel);
    return (double)(total_time / MEASURE_RUNS); // 返回毫秒
}
```

测试结果：测量十次取均值

```
正在初始化图像 (4096 x 4096)...
=== 图像卷积性能对比 (CPU vs GPU) ===
卷积核: 3x3 高斯模糊
----------------------------------------
正在运行 CPU 串行版本...
CPU 计算完成。
正在运行 GPU Global Memory 版本...
正在运行 GPU Constant Memory 版本...

=== 实验结果统计 ===
| Version             | Time (ms)    | Speedup     |
|---------------------|--------------|-------------|
| CPU Serial          | 685.00       | 1.00 x      |
| GPU (Global)        | 0.8622       | 794.48     x |
| GPU (Constant)      | 0.8239       | 831.45     x |
```

可以明显看到并行化后的加速比很高，并且使用常量内存优化以后，能够提到几十倍的加速效果。

附录

源代码：

以上全部小实验的源代码保存在一起完成提交。